

- **C++ as a tool for OOP(Object Oriented Programming)**

OOP(객체지향 프로그래밍)란? -

ADT

ADT : Abstract Data type

내장데이터

: (int, double, char, string 등)

- 객체지향프로그래밍
- 개체중심프로그래밍

- 추상적인 데이터형(Abstract Data Type, ADT)

- ^{개발자}~~사용자~~가 정의한 데이터 타입
 - 데이터와 그 데이터를 조작하는 함수를 포함
 - 추상적(abstract)의 의미는, 내장데이터 타입과 구분하기 위함.

실세계를 모방하는 데에는
procedure language < OOP
더 적합하다 !

- OOP가 ADT를 정의할 수 있도록 한다.
- OOP는 소프트웨어를 분석, 설계, 구현하는 방법 중의 한가지
 - 외형상 동일
 - 커다란 프로젝트에서 이점
- OOP는 특정언어와는 관련이 없다.

OOP방식

- 오브젝트는 데이터와 그 데이터를 조작할 수 있는 함수로 구성되어 있다.
- 데이터는 외부에 대해 감추어져 있어 반드시 인터페이스 함수를 통해서만 외부에 나타나게 된다.
- 물리적 오브젝트 – OOP가 더 자연스럽다.
- 오브젝트는 만질수 있는 것, 눈에 보이는 것만 말하지는 않는다.
- 관념적인 것도 포함. 도서관 업무같이 관념적인 것을 컴퓨터가 인식할 수 있는 수단으로 표현
- OOP규범 OOP 의 4가지 요소
 - 추상화(Abstraction)
 - 은닉화, 캡슐화(Encapsulation)
 - 상속성(Inheritance)
 - 다형성(Polymorphism)

OOP용어- 추상화(Abstraction)

pinpoint 가장 중요한 요소를 뽑아내는것

- 자세한 사항은 무시하고 본질적인 특징에 집중하는 과정
- 현실세계(물리적세계)의 사물을 데이터적인 측면과 기능적 측면을 통해서 정의하는 것

=====

코끼리:

특징1: 발이 4개

특징2: 코의 길이가 5미터 이내

특징3: 몸무게는 1톤 이상

특징4: 코를 이용해서 목욕을 함

특징5: 코를 이용해서 물건을 집음

=====

은행계좌의 추상화

```
private:
    hiding {
        class Account {
            int Anum;
            int balance;
            char name[100];

        public:
            void deposit(int );
            void withdraw(int );
            .
            .
        }
```

OOP용어- 은닉화, 캡슐화(Encapsulation)

- 추상화 단계에서 식별한 정보와 정보처리 방법을 하나의 단위로 묶는 것.
- 멤버데이터와 멤버함수를 하나의 단위로 결합하여 취급하는 것
- 멤버데이터는 감출 것 (데이터 감추기, Data Hiding)

클래스와 오브젝트

- 추상화와 캡슐화가 결합되어 하나의 새로운 데이터형을 정의
- 사용자가 정의한 데이터형을 추상적인 데이터형(ADT)이라고 함
- 추상적데이터형이란 내부에 감추어진 데이터(멤버변수)와 외부에서 그 데이터에 대해 조작할 수 있는 함수(멤버함수)로 이루어짐.
- 이러한 추상적인 데이터형을 클래스라 한다.
- 클래스에서 ~~하나의 오브젝트를 생성~~ ^{object “들” 을 생성}
- 클래스는 템플릿, 오브젝트는 인스턴스

타입과 인스턴스

- 타입(type)을 이용해 인스턴스(instance)를 생성할 수 있음
- 타입은 추상화된 것이고 인스턴스는 타입을 실체화한 구체적인 실체
- 타입과 인스턴스의 관계는 **일대다**(one-to-many) 관계
- 하나의 타입을 기반으로 많은 인스턴스를 생성할 수 있음

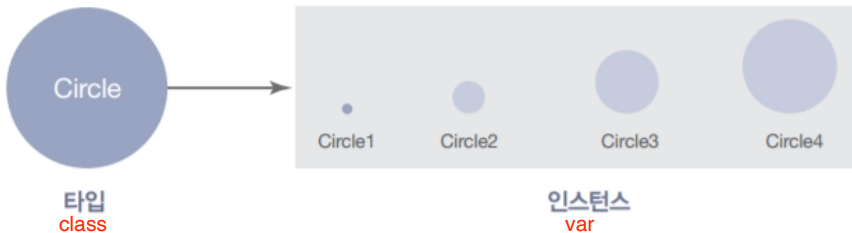


그림 7-1 타입과 인스턴스의 관계

속성과 행위

속성

member data

- 속성(attribute)은 인스턴스가 가지는 특징을 의미
- 컴퓨터 과학에서의 속성이란 우리가 관심 있어 하는 특성

행위

member function

- 인스턴스는 어떤 행위(behavior)를 가짐
- 행위란 어떤 인스턴스가 스스로 할 수 있는 작업 또는 연산을 말함

클래스와 객체

- C++은 클래스(class)라는 구문을 사용하여 타입(사용자 정의 자료형 ADT)을 생성
- 이러한 클래스를 기반으로 만든 인스턴스를 객체라고 부름. 또는 인스턴스라고 그냥 부르기도 함
- 객체 지향 프로그래밍에서 객체의 속성과 행위는 멤버 데이터와 멤버 함수로 생성

멤버 데이터

- 객체의 멤버 데이터는 속성을 표현하기 위한 변수를 의미

멤버 함수

- 프로그래밍에서 함수는 어떤 행위를 할 수 있는 기능의 모임
- 객체 지향 프로그래밍에서 객체의 행위는 멤버 함수를 사용해서 구현

비교

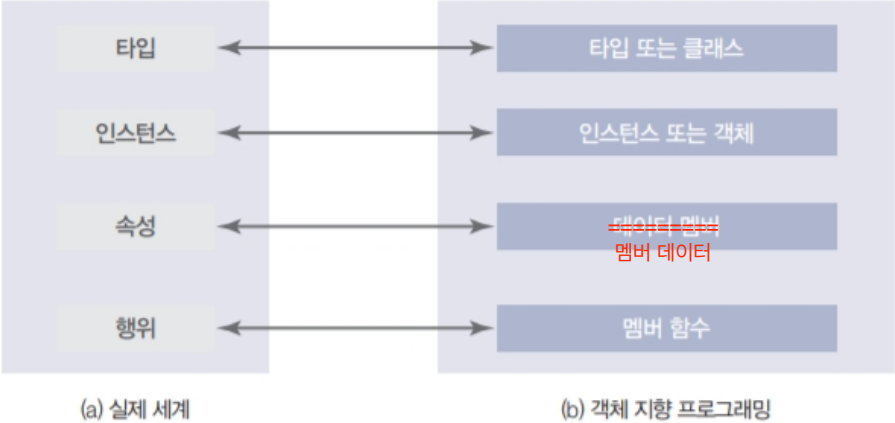


그림 7-2 실제 세계와 객체 지향 프로그래밍의 용어 비교

객체 지향과 클래스

- 새로운 자료형(타입)을 만들 때는 클래스 class 를 사용
- 객체 지향 프로그래밍을 할 때는 클래스 정의, 멤버 함수 정의, 애플리케이션(클래스를 기반으로 객체를 만들어서 사용하는 부분)이 필요

int main()

class definition (정의)

```
class Account{  
  
    int Anum;  
  
    int balance;  
    char name[100];  
  
    void deposit(int );  
    void withdraw(int );  
  
    -  
    -  
    .}
```

객체 지향과 클래스



그림 7-3 객체 지향 패러다임 C++ 프로그램의 세 부분

2개의 circle 객체 만들고 사용하기(1)

프로그램 7-1 2개의 circle 객체 만들고 사용하기

```
1  /*****
2  * A program to use a class in object-oriented programming      *
3  *****/
4  #include <iostream>
5  using namespace std;
6
7  /*****
8  * Class definition: the declaration of data members and member *
9  * functions of the class                                     *
10 *****/
11 class Circle
12 {
13     private:
14         double radius;
15     public:
16         double getRadius () const;
17         double getArea () const;
18         double getPerimeter () const;
19         void setRadius (double value);
20 };
```

2개의 circle 객체 만들고 사용하기(2)

프로그램 7-1 2개의 circle 객체 만들고 사용하기

```
21  /*****
22  * Members function definition. Each function declared in the *
23  * class definition section is defined in this section.      *
24  *****/
25  // Definition of getRadius member function
26  double Circle :: getRadius () const
27  {
28      return radius;
29  }
30  // Definition of getArea member function
31  double Circle :: getArea () const
32  {
33      const double PI = 3.14;
34      return (PI * radius * radius);
35  }
36  // Definition of getPerimeter member function
37  double Circle :: getPerimeter () const
38  {
39      const double PI = 3.14;
40      return (2 * PI * radius);
```

2개의 circle 객체 만들고 사용하기(3)

프로그램 7-1 2개의 circle 객체 만들고 사용하기

```
41 }
42 // Definition of setRadius member function
43 void Circle :: setRadius (double value)
44 {
45     radius = value;
46 }
47 /*****
48 * Application section: Objects are instantiated in this section. *
49 * Object use member functions to get or set their attributes. *
50 *****/
51 int main ( )
52 {
53     // Creating first circle and applying member functions
54     cout << "Circle 1: " << endl;
55     Circle circle1;
56     circle1.setRadius (10.0);
57     cout << "Radius: " << circle1.getRadius() << endl;
58     cout << "Area: " << circle1.getArea() << endl;
59     cout << "Perimeter: " << circle1.getPerimeter() << endl << endl;
60     // Creating second circle and applying member functions
```


2개의 circle 객체 만들고 사용하기(4)

프로그램 7-1 2개의 circle 객체 만들고 사용하기

```
61     cout << "Circle 2: " << endl;
62     Circle circle2;
63     circle2.setRadius (20.0);
64     cout << "Radius: " << circle2.getRadius() << endl;
65     cout << "Area: " << circle2.getArea() << endl;
66     cout << "Perimeter: " << circle2.getPerimeter();
67     return 0;
68 }
```

Run:

```
Circle 1:
Radius: 10
Area: 314
Perimeter: 62.8
Circle 2:
Radius: 20
Area: 1256
Perimeter: 125.6
```

클래스 정의(1)

- 새로운 타입을 만들려면 클래스 정의(class definition)를 작성
- 클래스 정의는 헤더, 본문, 세미콜론이라는 세 부분으로 구성
- 클래스 헤더(class header)는 class라는 키워드 뒤에 클래스의 이름을 붙여서 생성
- 클래스 본문(class body)은 멤버 데이터와 멤버 함수의 선언을 가진 블록(중괄호로 열고 닫히는 영역) 부분
- 마지막으로 세미콜론은 클래스 정의를 종료하겠다고 나타내는 부분

클래스 정의(2)

```
class Circle // Header
{
    private:
        double radius;    // Data member declaration
    public:
        *getter double getRadius() const; // Member function declaration
            double getArea() const; // Member function declaration
            double getPerimeter() const; // Member function declaration
        *setter void setRadius(double value); // Member function
            declaration
}; // A semicolon is needed at the end of class definition
```

radius 는 전역변수의 개념

private / public 이든

클래스 내부이므로 어디든 접근과 변경이 가능하다

클래스 정의(3)

멤버 데이터 선언

- 클래스 정의의 첫 번째 부분은 멤버 데이터를 선언하는 부분
- 멤버 데이터는 내장 자료형으로 만들 수도 있고, 이전에 정의한 다른 클래스로도 만들 수 있음
- 클래스의 속성은 멤버 데이터로 생성

클래스 정의(4)

멤버 함수 선언

- 클래스 정의의 두 번째 부분은 클래스의 멤버 함수를 선언하는 부분
- 클래스의 행위를 구현하는 데 사용할 함수를 생성
- 이전 장에서 함수를 만들 때 사용했던 함수 선언(프로토타입) 부분과 비슷

클래스 정의(5)

접근 제한자

- 접근 제한자(access modifier)는 접근 권한을 나타낼 때 사용
- 클래스에서 멤버 데이터와 멤버 함수를 선언하면 기본적으로 private 접근 제한자가 붙음
- private 접근 제한자가 붙으면 해당 요소로부터 값을 추출할 수도 없고, 값을 변경할 수도 없음

클래스 정의(6)



그림 7-4 접근 제한자

표 7-1 접근 제한자의 멤버 접근 범위

접근 제한자	같은 클래스에서의 접근	서브 클래스에서의 접근	모든 곳으로부터의 접근
private	가능	불가능	불가능
protected	가능	가능	불가능
public	가능	가능	가능

클래스 정의(7)

멤버 데이터와 접근 제한자

- 멤버 데이터에는 일반적으로 `private`을 적용
- 기본적으로 `private`이 붙지만, 강조하기 위해서 `private`을 붙여서 사용
- `private`이 적용된 멤버는 멤버 함수를 통해서만 접근할 수 있음

멤버 함수와 접근 제한자

- 멤버 데이터에 접근해서 어떤 작업을 하려면, 애플리케이션에서 멤버 함수를 사용할 수 있어야 하며 따라서 일반적으로 멤버 함수는 `public`을 적용

클래스 정의(8)

그룹 접근 제한자

- 이전의 클래스 정의 코드를 보면 `private` 키워드와 `public` 키워드를 한 번씩만 사용
- 접근 제한자는 각각의 것 앞에 붙이는 것이 아니라 그룹 단위로 붙임

멤버 함수 정의(1)

- 멤버 함수 선언은 단순히 함수의 프로토타입을 적은 것이며 따라서 별도의 정의가 필요

```
double Circle :: getRadius() const
{
    return radius;
}
```

```
double Circle :: getArea() const
{
    const double PI = 3.14;
    return (PI * radius * radius);
}
```

멤버 함수 정의(2)

```
double Circle :: getPerimeter() const
{
    const double PI = 3.14;
    return (2 * PI * radius);
}
```

setter 함수

```
void Circle :: setRadius(double value)
{
    radius = value;
}
```

멤버 함수 정의(3)

- 멤버 함수의 정의는 이전에 살펴보았던 기본적인 함수 정의와 비슷하지만 2가지 차이점이 있음
- 첫 번째는 멤버 함수에는 한정자 `const` 가 있다는 것
- 두 번째는 멤버 함수에는 앞에 클래스 이름이 붙는다는 것
- C++도 멤버 함수 이름 앞에 클래스 이름과 클래스 스코프 기호(`::`)를 성처럼 붙여야 함

표 7-2 클래스 스코프

그룹	이름	연산자	표현식	우선 순위	결합방향
기본	클래스 스코프	::	클래스::이름	19	→

멤버 함수 정의(4)

```
class Circle
{
    ...
    public:
        double getRadius() const;
    ...
}
```

(a) 클래스 스코프 연산자가 필요 없는 상황

```
double Circle::getRadius()const;
{
    ...
}
...
```

(b) 클래스 스코프 연산자가 필요한 상황

그림 7-5 클래스 정의와 멤버 함수 정의에서의 클래스 스코프 연산자 사용 여부

애플리케이션(1)

- 클래스 정의와 멤버 함수를 정의했다면 `main` 함수 부분에서 이를 인스턴스화하고 사용

애플리케이션(2)

객체 인스턴스화

- 멤버 함수를 사용하려면, 반드시 다음과 같은 방법으로 객체를 인스턴스화

```
Circle circle1;
```

객체에 연산 적용

- 인스턴스화했다면 멤버 함수로 정의했던 연산을 사용할 수 있게 됨

```
circle1.setRadius(10.0);  
cout << "Radius: " << circle1.getRadius() << endl;  
cout << "Area: " << circle1.getArea() << endl;  
cout << "Perimeter: " << circle1.getPerimeter() << endl <<  
endl;
```

구조체

- 가끔 다른 사람이 만든 소스 코드 등을 분석할 때, C언어 때부터 있었던 구조체를 C++에서 볼 수도 있음
- 구조체의 모든 멤버는 기본적으로 public
- 반대로 클래스의 모든 멤버는 기본적으로 private
- 따라서 구조체를 다음과 같이 클래스로 만들 수 있음

```
struct
{
    string first;
    char middle;
    string last;
};
```

```
class
{
    public:
        string first;
        char middle;
        string last;
};
```


생성자와 소멸자(1)

- 객체가 멤버 데이터를 갖고 어떤 작업을 하려면, 객체를 만든 뒤 멤버 데이터를 초기화하는 작업이 필요
- 객체는 생성자(constructor)라고 부르는 특별한 멤버 함수가 호출될 때 생성됩니다. 따라서 생성자 내부에서 초기화를 하면 편리

생성자와 소멸자(2)

destructor

- 또한 객체가 더 이상 필요가 없어지는 경우, 객체가 차지하고 있는 메모리를 비워줘야 함(메모리 재활용).
- 이때는 소멸자(destructor)라고 부르는 특별한 멤버 함수가 호출되어 소멸자 내부에서 객체를 정리하는 작업

생성자와 소멸자(3)



그림 7-6 객체의 라이프 사이클

`char*` 은 생성자에서 메모리 확보를 해주어야한다

생성자(1)

- 생성자는 객체를 생성하는 특별한 멤버 함수
- 생성자 내부에서 객체의 멤버 데이터를 초기화
- 생성자는 2가지 특징이 있으며 첫 번째는 리턴값이 없으며, 두 번째는 이름이 클래스의 이름과 같다는 것
- 생성자는 크게 매개변수가 있는 생성자, 기본 생성자, 복사 생성자라는 3가지로 구분

생성자(2)

생성자 선언

- 생성자는 클래스의 멤버 함수이므로 클래스 정의에서 선언
- 멤버 데이터를 초기화하는 변경 작업을 하므로 `const` 한정자를 붙일 수 없음

```
class Circle
{
    . . . 함수 시그니처 : function name, parameter 의 갯수, parameter 의 DataType
    public: function overloading 때문에 가능하다
        Circle(double radius); // Parameter Constructor
        Circle(); // Default Constructor
        Circle(const Circle& circle); // Copy Constructor
    . . .
}
```

생성자(3)

생성자(3) - 추가 라는 ppt page 있음 사진 찍어놨으니 추가해야함

Circle c1; 매개변수가 있는 생성자(parameter constructor)

- 일반적으로 멤버 데이터를 지정된 값으로 초기화하기 위해서 매개변수가 있는 생성자를 사용

Circle c2(5.8);

기본 생성자(default constructor)

- 기본 생성자는 매개변수가 없는 생성자를 의미

Circle c3(c2);

복사 생성자(copy constructor)

- 복사 생성자로 객체를 복사하면 원본과 복사된 사본이 같은 값을 갖는 다른 객체로 생성

생성자(4)

생성자 정의

- 생성자는 특별한 멤버 함수
- 생성자는 리턴값을 가질 수 없으며, 이름은 클래스의 이름과 같음

```
// Definition of a parameter constructor
Circle :: Circle (double rds)
: radius(rds) // Initialization list
{
    // Any other statements
}
```

```
// Definition of a default constructor
Circle :: Circle()
: radius(0.0) // Initialization list.
{
    // Any other statements
}
```

```
// Definition of a parameter constructor
Circle :: Circle (double rds)
{
    radius = rds;
    cout << "parameter있는 생성자" <
}
```

```
// Definition of a default constructor
Circle :: Circle()
{
    radius = 0.0;
    cout << "parameter없는 생성자" <
}
```

생성자(5)

```
// Definition of a copy constructor
Circle :: Circle(const Circle& cr)
: radius(cr.radius) // Initialization list
{
    // Any other statements
}
```

```
// Definition of a copy constructor
Circle :: Circle(const Circle& cr)
{ 객체의 크기가 크기때문에 copy 하지 말고 share 하자는 pass-by-reference
  radius = cr.radius;
  cout << "복사 생성자" << endl;
}
```


소멸자(1)

- 생성자와 마찬가지로 소멸자도 2가지 특징이 있음
- 첫 번째는 소멸자의 이름은 클래스 이름 앞에 물결 기호(~)가 붙은 형태
- 두 번째는 생성자와 마찬가지로 소멸자는 리턴값을 가질 수 없음
- 소멸자는 객체가 스코프를 벗어나는 등의 상황에 자동적으로 호출

소멸자(2)

소멸자 선언

- 다음은 클래스 정의 내부에서 소멸자를 정의하는 예시

```
class Circle
{
    ...
    public:
        ...
        ~Circle();    // Destructor
}
```

소멸자(3)

소멸자 정의

- 소멸자도 다른 멤버 함수처럼 정의
- 다만 이름 앞에 물결 기호(~)가 있다는 특징이 있으므로 주의

```
// Definition of a destructor
Circle :: ~Circle()
{
    // Any statements as needed
}
```

생성자와 소멸자

매개변수가 있는 생성자

```
Circle circle1 (5.1);
```

기본 생성자

```
Circle circle2;
```

괄호 없어도 됨

복사 생성자

```
Circle circle3 (aCircle );
```

aCircle은 이미 존재하는 다른 객체

소멸자

시스템에 의해 호출됨

사용자가 호출하는 것이 아님

그림 7-7 생성자와 소멸자의 호출

생성자와 소멸자

표 7-3 변수 생성과 클래스 객체 생성 비교

멤버	클래스 자료형	내장 자료형
매개변수가 있는 생성자	Circle circle1(10.0);	double x1 = 10.0;
기본 생성자	Circle circle2;	double x2;
복사 생성자	Circle circle3(circle1)	없음
소멸자	사용자가 호출하지 않음	사용자가 호출하지 않음

필수 멤버 함수

매개변수가 있는 생성자
기본 생성자

그룹 1

복사 생성자

그룹 2

소멸자

그룹 3

❗ 그룹별로 함수가 하나씩 필요

만약 만들지 않으면, 시스템이 해당 그룹에 속하는 함수를 자동으로 생성

별로 안중요

그림 7-8 생성자와 소멸자 그룹

필수 멤버 함수 그룹(1)

- 그룹 1은 매개변수가 있는 생성자와 기본 생성자로 구성
- 이러한 생성자가 하나 이상 있어야 하며 2개를 모두 만들어도 상관 없음
- 만약 둘 다 만들지 않으면, 시스템은 합성 기본 생성자(synthetic default constructor)라는 것을 생성
별로 안중요
- 합성 기본 생성자는 멤버 데이터를 쓰레기 값으로 초기화

필수 멤버 함수 그룹(2)

- 그룹 2는 복사 생성자로 구성
- 클래스에는 하나의 복사 생성자가 있어야 함
- 만약 만들지 않으면 시스템이 합성 복사 생성자(synthetic copy constructor)를 생성
별도안중요
- 하지만 일반적으로 복사 생성자는 직접 구현해서 원하는 형태로 만드는 것이 좋음

필수 멤버 함수 그룹(3)

- 그룹 3은 소멸자로 구성
- 클래스에는 하나의 소멸자가 있어야 함
- 만약 만들지 않으면, 시스템이 합성 소멸자(synthetic destructor)를 생성
- 일반적으로 합성 소멸자는 우리가 ^{별로안중요} 원하는 동작을 하지 않으므로 직접 구현하는 것이 좋음

Circle 클래스 완성(1)

프로그램 7-2 Circle 클래스 완성

```
1  /*****
2  * A program to use a class in object-oriented programming      *
3  *****/
4  #include <iostream>
5  using namespace std;
6
7  /*****
8  * Class Definition:                                           *
9  * declaration of parameter constructor, default constructor, *
10 * copy constructor, destructor, and other member functions    *
11 *****/
12 class Circle
13 {
14     private:
15         double radius;
16     public:
17         Circle (double radius); // Parameter Constructor
18         Circle (); // Default Constructor
19         ~Circle (); // Destructor
20         Circle (const Circle& circle); // Copy Constructor
```

Circle 클래스 완성(2)

프로그램 7-2 Circle 클래스 완성

```
21         void setRadius (double radius); // Mutator
22         double getRadius () const; // Accessor
23         double getArea () const; // Accessor
24         double getPerimeter () const; // Accessor
25     };
26     /*****
27     * Member Function Definition:
28     * Definition of parameter constructor, default constructor,
29     * copy constructor, destructor, and other member functions
30     *****/
31     // Definition of parameter constructor
32     Circle :: Circle (double rds)
33     : radius (rds)
34     {
35         cout << "The parameter constructor was called. " << endl;
36     }
37     // Definition of default constructor
38     Circle :: Circle ()
39     : radius (0.0)
40     {
```

Circle 클래스 완성(3)

프로그램 7-2 Circle 클래스 완성

```
41     cout << "The default constructor was called. " << endl;
42 }
43 // Definition of copy constructor
44 Circle :: Circle (const Circle& circle)
45 : radius (circle.radius)
46 {
47     cout << "The copy constructor was called. " << endl;
48 }
49 // Definition of destructor
50 Circle :: ~Circle ()
51 {
52     cout << "The destructor was called for circle with radius " ;
53     cout << endl;
54 }
55 // Definition of setRadius member function
56 void Circle :: setRadius (double value)
57 {
58     radius = value;
59 }
60 // Definition of getRadius member function
```

Circle 클래스 완성(4)

프로그램 7-2 Circle 클래스 완성

```
61 double Circle :: getRadius () const
62 {
63     return radius;
64 }
65 // Definition of getArea member function
66 double Circle :: getArea () const
67 {
68     const double PI = 3.14;
69     return (PI * radius * radius);
70 }
71 // Definition of getPerimeter member function
72 double Circle :: getPerimeter () const
73 {
74     const double PI = 3.14;
75     return (2 * PI * radius);
76 }
77 /*****
78 * Application :
79 * Creating three objects of class Circle (circle1, circle2,
80 * and circle3) and applying some operation on each object *
```

Circle 클래스 완성(5)

프로그램 7-2 Circle 클래스 완성

```
81  *****/
82  int main ()
83  {
84  // Instantiation of circle1 and applying operations on it
85  Circle circle1 (5.2);
86  cout << "Radius: " << circle1.getRadius() << endl;
87  cout << "Area: " << circle1.getArea() << endl;
88  cout << "Perimeter: " << circle1.getPerimeter() << endl << endl;
89  // Instantiation of circle2 and applying operations on it
90  Circle circle2 (circle1);
91  cout << "Radius: " << circle2.getRadius() << endl;
92  cout << "Area: " << circle2.getArea() << endl;
93  cout << "Perimeter: " << circle2.getPerimeter() << endl << endl;
94  // Instantiation of circle3 and applying operations on it
95  Circle circle3;
96  cout << "Radius: " << circle3.getRadius() << endl;
97  cout << "Area: " << circle3.getArea() << endl;
98  cout << "Perimeter: " << circle3.getPerimeter() << endl << endl;
99  // Calls to destructors occur here
100 return 0;
```

Circle 클래스 완성(6)

프로그램 7-2 Circle 클래스 완성

```
101 }
```

Run:

The parameter constructor was called.

Radius: 5.2

Area: 84.9056

Perimeter: 32.656

The copy constructor was called.

Radius: 5.2

Area: 84.9056

Perimeter: 32.656

The default constructor was called.

Radius: 0

Area: 0

Perimeter: 0

The destructor was called for circle with radius: 0

The destructor was called for circle with radius: 5.2

The destructor was called for circle with radius: 5.2